Презентация на семинар AGI

Page · Tag · 2 обратные ссылки

https://nsu-ai.ktalk.ru/cfsii65x4r8t

Онтологический подход к построению формальных семантик языков программирования

Ануреев Игорь Сергеевич

кандидат физико-математических наук,

старший научный сотрудник Института систем информатики им. А.П. Ершова СО РАН

старший научный сотрудник Института автоматики и электрометрии СО РАН

доцент Новосибирского государственного университета

Зачем нужна формальная семантика ЯП

- Для анализа свойств программ
- Для оптимизации программ
- Для компиляторов
- Для порождения тестов
- Для сравнения ЯП на смысловом уровне
- Для прототипирования новых ЯП

Выбор семантик для рассмотрения

- Операционная семантика → run-time verification
- Аксиоматическая семантика → formal program verification, symbolic execution

Требования к DSL для построения формальных семантик

- Языковые средства спецификации онтологий ЯП
    - Онтологические модели программ и вместо синтаксических представлений (AST) и
    - Онтологические модели состояний программ вместо разношерстного задания состояний на разных ЯП и математических формализмах
    - Онтология ЯП = онтология синтаксических и семантических конструкций ЯП + онтология состояния программ
    - Онтология ⇒ универсальность, гибкость, расширяемость, модульность, комбинируемость, ... (при спецификации семантики)
- Языковые средства трансформации онтологических моделей ⇒ построение спецификаций семантик
- Выполнимость ⇒ динамические и статические средства верификации
- Встроенный DSL в ЯП (eDSL) ⇒ возможность использования существующей инфраструктуры этого ЯП (IDE, редактирование, отладка, ...)
- Компактность
- Расширяемость

ABML (Attribute-based Modelling Language)

- Лексическое расширение Common Lisp ⇒ eDSL, Расширяемость
- ЯП — частный случай дискретной динамической системы (далее системы)
- Основные сущности (онтология ABML)
    - Для спецификации легковесных онтологий конструкций ЯП
        - Объекты ⇒ элементы и подсистемы систем = individuals (***атрибуты***, ***мутабельные***, ***константные***)
        - Атрибуты ⇒ свойства и отношения объектов = object и value properties (***имя***, ***объект***, ***значение***, ***тип***)
        - Объектные типы ⇒ классы объектов = concepts, classes (***мутабельные***, ***константные***)
    - Для спецификации состояний программы и переходов между состояниями

- Аспекты ⇒ способы работы с объектами и классами объектов (***имя***, ***выполнение аспекта***, ***запуск аспекта***, ***аспектный контекст***)
- Агенты ⇒ информация (знание) о части состояния системы (***локальное состояние***, ***объектный тип***)
- Окружение ⇒ информация (знание) об общих частях состояния системы (***агенты***)
- Стадии ⇒ отдельные этапы выполнения аспекта (***аспект***)

Объектные типы

- (mot $n\ ad_1\ ...\ ad_r$) или (cot $n\ ad_1\ ...\ ad_r$)
- Объявление атрибутов ad (ограничения на атрибуты) имеет вид
  - :av $a\ v$
  - :at $a\ t$
  - :atv $a\ t\ v$
  - :amap $t_1\ t_2$
- Примеры

```lisp
(mot "type name 1" :at "counter" int)
(mot "type name 2" :at "counter"
    (lambda x (and (is-instance x int) (<= 0 x) (<= x 7))))
(mot "if statement" :at "condition" "expression"
    :at "then" "statement" :at "else" "statement")
```

Объекты

- (mo $t\ ad_1\ ...\ ad_r$) или (co $t\ ad_1\ ...\ ad_r$)
- Объявление атрибутов ad (ограничения на атрибуты) имеет вид
  - :av $a\ v$
- Принцип открытости мира
- (mo $ad_1\ ...\ ad_r$) или (co $ad_1\ ...\ ad_r$) для типов mot и cot
- Предопределенные функции над объектами

    - (otype o)
    - (is-instance o t)
    - (attributes o)

Базовые функции работы с атрибутами

- (att t ad_1 ... ad_r)
- (aget o a)
    - (aget o a_1 ... a_n)
    - (aget o (aseq a_1 ... a_n))
    - (aget o (aseql e))
- (aset o a v)
    - (aset o a_1 ... a_n v)
    - (aget o (aseq a_1 ... a_n) v)
    - (aget o (aseql e)) v)
    - (aset o :av a_1 v_1 ... a_n v_n)

Механизм сопоставления с образцом

- (match c_1 ... c_r) или (nmatch c_1 ... c_r)
- Клозы атрибутов
    - :av e a v
    - :at e a t
    - :ap e a p
- Клозы выражений
    - :v e v
    - :t e t
    - :p e p
- Клозы действий
    - :do e_1 ... e_n
    - :exit e_1 ... e_n
    - :pred e

Аспектные контексты

- Специфицирует контекст, в котором выполняется аспект
- Определяется как константный объект
    - "aspect" — имя аспекта
    - "instance" — объект, над которым выполняется аспект
    - "agent" — агент, к которому обращается аспект
    - "env" — окружение, к которому обращается аспект
    - "stage" — имя стадии выполнения аспекта
    - любые другие атрибуты
- выполнить аспект в контексте = выполнить аспектный контекст

Операции над аспектными контекстами

- (clear-context ac)
- (aspect a :type t :stage st :context ac :instance i :agent ag :env env :do e_1 ... e_n)
- Аспектный префикс = задает условия выполнения аспекта
- Аспектное тело = описывает выполняемые действия
- Расширение аспектного префикса :ap $e\ a\ p$ и :p $e\ p$
- Сокращение: :ap $a\ p \equiv$:ap $ac\ a\ p$
- Вычисление аспектного тела = (progn e_1 ... e_n (acontext-next ac))
- Остановка

```lisp
(cot "stop next acontext" :at "type" string :at "acontext" cot)
```

- Запуск = (eval-acontext ac)

Агенты и окружение

- Обязательные атрибуты агентов
    - value

- Обязательные атрибуты окружения
    - "agents" : (listt "agent")
    - "acontexts" : (cot :amap "agent" (listt "acontext"))

Операции над окружением

- (push-acontext ac) — добавить
- (pop-acontext ac) — взять
- (peek-acontext ac) — считать
- (update-push-acontext ac ...) $\equiv$ (push-acontext (aset ac ...))
- (clear-update-push-acontext ac ...) $\equiv$ (update-push-acontext (clear-acontext ac ...))

Методика построения операционной семантики

- Пусть PL — целевой ЯП

1. Построить онтологию PL-конструкций (онтологию PL) как набор типов языка ABML
2. Разработать алгоритм трансляции PL-конструкций в их онтологические модели
3. Реализовать транслятор на основе этого алгоритма
4. Определить типы для агентов и окружения
5. Определить операционную семантику онтологических моделей PL-конструкций как набор деклараций аспекта "opsem"

Фиксация модельного языка PL

- PL содержит все конструкции из следующего C-кода

```C-like
{
    int x = 0;
    int y;
    int *p = &x;
```

```
    if (*p == 0) y = 2;
}
```

Онтология PL — Имена, константы, типы

Lisp

```
(typedef "name" (uniont symbol string))
(typedef "variable" "name")
(typedef "constant" ("int constant" "null constant"))
(typedef "int constant" int)
(typedef "null constant" (enumt "null"))
(typedef "type" (uniont "int type" "pointer type1"))
(typedef "int type" (enumt "int"))
(mot "pointer type1" :at 1 "type")
```

Онтология PL — Выражения

Lisp

```
(mot "variable access" :at "variable" "variable")
(mot "&1" :at 1 "variable")
(mot "*1" :at 1 "variable")
(mot "1=2" :at 1 "variable" :at 2 "expression")
(mot "1==2" :at 1 "expression" :at 2 "expression")
(typedef "expression" (uniont "variable access" "constant" "&1"
    "*1" "1=2" "1==2"))
```

Онтология PL — Операторы

Lisp

```
(mot "1;" :at 1 "expression")
(mot "{1}" :at 1 (listt "statement"))
(mot "if12" :at 1 "expression" :at 2 "statement")
(mot "if12else3" :at 1 "expression" :at 2 "statement" :at 3
"statement")
(mot "variable declaration" :at "type" "type" :at "name" "variable"
```

```
    :at "initializer" "expression")
(typedef "statement" (uniont "1;" "{1}" "if12" "variable declaration"))
```

Алгоритм трансляции PL-конструкций в их онтологические модели и его реализация

- Практически взаимно-однозначное отображение

```lisp
(mo "{1}" :av 1 (list
    (mo "variable declaration" :av "type" "int" :av "name" "x"
        :av "initializer" 0)
    (mo "variable declaration" :av "type" "int" :av "name" "y")
    (mo "variable declaration" :av "type" (mo "pointer type1" :av 1
"int")
        :av "name" "p" :av "initializer"
            (mo "&1" :av 1 (mo "variable access" :av "variable" "x")))
    (mo "if12" :av 1 (mo "1==2" :av 1 (mo "*1" :av 1 "p") :av 2 0)
        :av 2 (mo "1;" :av 1 (mo "1=2" :av 1
            (mo "variable access" :av "variable" "y") :av 2 2)))))
```

Онтология семантических PL-конструкций

```lisp
(mot "location" :at "value" "C value")
(typedef "C value" (uniont "constant" "location"))

(mo "location" :av "value" (mo "location" :av "value" 2))
```

Спецификация среды и агентов

```lisp
(mot "env" :at "agents" (listt "agent")
    :at "acontexts" :amap "agent" (listt "acontext"))
(mot "agent"
    :at "location" (cot :amap "variable" "location")
```

```
    :at "value" "c value")
(typedef "acontext" cot)
```

Семантика доступа к переменным как пример шаблона проектирования "Одна стадия"

Lisp

```
(aspect "opsem" :type "variable access" :instance i :agent a
    :do (aget a "location" (aget i "variable") "value")
```

Семантика операции равенства как пример шаблона проектирования "Последовательность стадий"

Lisp

```
(aspect "opsem" :type "1==2" :acontext ac
    :do (update-push-acontext ac :av "stage" "execute 1"))

(aspect "opsem" :type "1==2" :stage "execute 1"
    :acontext ac :instance i :ap i 1 e1
    :do (update-push-acontext ac :av "stage" "execute 2"))
        (clear-update-eval-acontext ac :av "instance" e1))

(aspect "opsem" :type "1==2" :stage "execute 2" :value v1
    :acontext ac :instance i :ap i 2 e2
    :do (update-push-acontext ac :av "stage" "compare" :av "v1" v1)
    (clear-update-eval-acontext ac :av "instance" e2))

(aspect "opsem" :type "1==2" :stage "compare" :ap "v1" v1 :value v2
    :do (equal v1 v2)))
```

Семантика условного оператора как пример шаблона проектирования "Разбор случаев"

```lisp
(aspect "opsem" :type "if12else3" :acontext ac
    :do (update-push-acontext ac :av "stage" "execute 1"))

(aspect "opsem" :type "if12else3" :stage "execute 1"
    :acontext ac :instance i :ap i 1 e1
    :do (update-push-acontext ac :av "stage" "check 1"))
        (clear-update-eval-acontext ac :av "instance" e1))

(aspect "opsem" :type "if12else3" :stage "check 1" :value v1
    :acontext ac :instance i :ap i 2 s2 :ap i 2 s3
    :match :pred (or (equal v1 0) (equal v1 0.0) (equal v1 "null"))
        :exit (clear-update-eval-acontext ac :av "instance" s2)
        :do (clear-update-eval-acontext ac :av "instance" s3))
```

Семантика блока как пример шаблона проектирования "Итерация по стадии"

```lisp
(aspect "opsem" :type "{1}" :acontext ac :instance i :ap i 1 sts
    :match :av sts nil
    :exit (update-push-acontext ac :av "stage" "block iteration"
    :av "statements" sts :av "current" 0 :av "bound" (length sts)))

(aspect "opsem" :type "{1}" :stage "block iteration" :acontext ac
    :av "statements" sts :av "current" j :av "bound" n
    :match :v (< j n) T
    :exit (update-push-acontext ac :av "current" (+ j 1))
    (match :ap ac "stage" "block iteration" :ap ac "statements" sts
        (clear-update-eval-acontext ac :av "instance" (nth j sts))))
```

Методика построения аксиоматической семантики

- Пусть PL — целевой ЯП, L — логический язык задания требований, APL — язык аннотированных PL-конструкций с вкраплениями аннотаций на языке L, CVCL — язык контексто-обеспеченных условий корректности

1. Построить онтологии языков PL, L, APL, CVCL как наборы типов языка ABML
2. Разработать алгоритмы трансляции конструкций языков PL, L, APL в их онтологические модели
3. Реализовать соответствующие трансляторы на основе этих алгоритмов
4. Определить типы для агентов и окружения
5. Определить аксиоматическую семантику онтологических моделей PL-конструкций как набор деклараций аспекта "axsem"
6. Разработать алгоритмы трансляции онтологических моделей языка CVCL в системы машинной поддержки доказательства (SMT-solvers, proof assistants, automated provers) — Либо разработать программный интерфейс для взаимодействия с этими системами как библиотеку функций языка ABML
7. Реализовать соответствующие трансляторы на основе этих алгоритмов — Либо реализовать эти функции

Фиксация логического языка L

- Термы
  - PL-выражения без побочного эффекта — Либо только переменные и константы
  - $f(t_1, ..., t_n)$ — Либо $t(t_1, ..., t_n)$ в логике высшего порядка
  - lambda x . t — Возможно lambda x_1 ... x_n . t
  - (t)
- Формулы
  - t_1 == t_2
  - true, false
  - not u_1, u_1 and ... and u_n, u_1 or ... or u_n
  - forall x . u, exists x . u — Возможно forall x_1 ... x_n . u, exists x_1 ... x_n. u
  - $p(t_1, ..., t_n)$ — Либо $t(t_1, ..., t_n)$ в логике высшего порядка (формулы — частный случай термов)
  - (u)

Фиксация аннотированного модельного

языка PL

- PL содержит все конструкции из следующего аннотированного C-кода

```C-like
{
    /*@ assume true */
    int x = 0;
    int y;
    int *p = &x;
    if (*p == 0) y = 2;
    /*@ assert y = 2 */
}
```

Варианты возможных аннотаторов

- assume u
- assert u
- inv u
- inv-loop u

Фиксация языка CVCL (Context-carrying Verification Conditions Language)

- Определения предикатов (pd)
    - $p(x_1, ..., x_n) := u$
- Определения функций (fd)
    - $f(x_1, ..., x_n) := t$
- Обеспеченные контекстом условия корректности
    - $pd_1, ..., pd_n \ + fd_1, ..., fd_m \Rightarrow u$

Возможные варианты конструкциий языка CVCL

- Определения предикатов
- Определения функций
- Определения теорем и лемм
- Определения аксиом
- Определения тактик
- Определения скриптов доказательства

ИКР для взаимодействия с CVCL-конструкциями

- МегаХаммер
- Конфигуратор систем поддержке машинного доказательства
- План доказательства
- Операции композиции планов
- Универсальный доказатель

Онтология L

Lisp

```lisp
(typedef "term" (uniont "expression" "fcall12"))

(typedef "formula" (uniont "logical constant" "not1" "or1" "and1"
"log1==2" "forall12" "exists12" "pcall12"))

(mot "logical constant" (enumt "true" "false"))

(mot "not1" :at 1 "formula")
(mot "or1" :at 1 (listt "formula"))
(mot "and1" :at 1 (listt "formula"))

(mot "lambda12" :at 1 "variable" :at 2 "term")
```

```
(mot "lambda12" :at 1 (listt "variable") :at 2 "term")
(mot "forall12" :at 1 "variable" :at 2 "formula")
(mot "forall12" :at 1 (listt "variable") :at 2 "formula")
(mot "exists12" :at 1 "variable" :at 2 "formula")
(mot "exists12" :at 1 (listt "variable") :at 2 "formula")

(typedef "quantor name" "name")
(mot "quantor123" :at 1 "quantor name" :at 2 "variable" :at 3 "term")
(mot "quantor123" :at 1 "quantor name" :at 2 (listt "variable") :at 3
"term")

(mot "log1==2" :at 1 "term" :at 2 "term")

(mot "pcall12" :at 1 "predicate name" :at 2 (listt "term"))
(mot "fcall12" :at 1 "function name" :at 2 (listt "term"))

(typedef "predicate name" "name")
(typedef "function name" "name")
```

Онтология APL — Имена, константы, типы

Lisp

```
(typedef "name" (uniont symbol string))
(typedef "variable" "name")
(typedef "constant" ("int constant" "null constant"))
(typedef "int constant" int)
(typedef "null constant" (enumt "null"))
(typedef "type" (uniont "int type" "pointer type1"))
(typedef "int type" (enumt "int"))
(mot "pointer type1" :at 1 "type")
```

Онтология APL — Выражения

Lisp

```
(mot "variable access" :at "variable" "variable")
(mot "&1" :at 1 "variable")
(mot "*1" :at 1 "variable")
```

```
(mot "1=2" :at 1 "variable" :at 2 "expression")
(mot "1==2" :at 1 "expression" :at 2 "expression")
(typedef "expression" (uniont "variable access" "constant" "&1"
    "*1" "1=2" "1==2"))
```

Онтология APL — Операторы

Было

Lisp

```
(mot "1;" :at 1 "expression")
(mot "{1}" :at 1 (listt "statement"))
(mot "if12" :at 1 "expression" :at 2 "statement")
(mot "if12else3" :at 1 "expression" :at 2 "statement" :at 3
"statement")
(mot "variable declaration" :at "type" "type" :at "name" "variable"
    :at "initializer" "expression")
(typedef "statement" (uniont "1;" "{1}" "if12" "variable declaration"))
```

Стало

Lisp

```
...
(typedef "statement" (uniont "1;" "{1}" "if12" "variable declaration"
    "assume1" "assert1" "inv1"))
(mot "assume1" :at 1 "formula")
(mot "assert1" :at 1 "formula")
(mot "inv1" :at 1 "formula")
```

Онтология CVCL

Lisp

```
(mot "cvc" :at "pdefinitions" (listt "pdefinition123")
    :at "fdefinitions" (listt "fdefinition123")
    :at "vc" "formula")
```

```
(mot "pdefinition123" :at 1 "predicate name" :at 2 (listt "variable")
    :at 3 "formula"))
(mot "fdefinition123" :at 1 "function name" :at 2 (listt "variable")
    :at 3 "term"))
```

Спецификация среды и агентов

Было

Lisp

```
(mot "env"
    :at "agents" (listt "agent")
    :at "acontexts" :amap "agent" (listt "acontext"))
(typedef "acontext" cot)

(mot "agent"
    :at "value" "c value"
    :at "location" (cot :amap "variable" "location"))
```

Стало

Lisp

```
(mot "env"
    :at "agents" (listt "agent")
    :at "acontexts" :amap "agent" (listt "acontext")
    :at "cvcs" (listt "cvc"))) ; список порожденных УК
(typedef "acontext" cot)

(mot "agent"
    :at "value" "term" ; символьное значение
    :at "precondition" (listt "formula") ; предусловие
    :at "state" "variable name")) ; имя переменной, хранящей текущее
состояние
```

Семантика доступа к переменным как пример шаблона проектирования "Одна

стадия"

Было

```lisp
(aspect "opsem" :type "variable access" :instance i :agent a
    :do (aget a "location" (aget i "variable") "value")
```

Стало

```lisp
(aspect "axsem" :type "variable access" :instance i :agent a :ap a
"state" st
    :do (mo "fcall" "loc2val"
        :av (list st (mo "fcall" "var2loc"
            :av (list st (aget i "variable"))))
```

Семантика операции равенства как пример шаблона проектирования "Последовательность стадий"

```lisp
(aspect "axsem" :type "1==2" :acontext ac
    :do (update-push-acontext ac :av "stage" "execute 1"))

(aspect "axsem" :type "1==2" :stage "execute 1"
    :acontext ac :instance i :ap i 1 e1
    :do (update-push-acontext ac :av "stage" "execute 2"))
        (clear-update-eval-acontext ac :av "instance" e1))

(aspect "axsem" :type "1==2" :stage "execute 2" :value v1
    :acontext ac :instance i :ap i 2 e2
    :do (update-push-acontext ac :av "stage" "compare" :av "v1" v1)
    (clear-update-eval-acontext ac :av "instance" e2))

(aspect "axsem" :type "1==2" :stage "execute 1" :ap "v1" v1 :value v2
    :do (mo "1==2" :av 1 v1 :av 2 v2))
```

```
(aspect "axsem" :type "1==2" :stage "execute 1" :ap "v1" v1 :value v2
    :do (mo "fcall" :av 1 "log==" :av 2 (list v1 v2)))
```

Семантика условного оператора как пример шаблона проектирования "Разбор случаев"

Lisp

```
(aspect "axsem" :type "if12else3" :acontext ac
    :do (update-push-acontext ac :av "stage" "execute 1"))

(aspect "axsem" :type "if12else3" :stage "execute 1"
    :acontext ac :instance i :ap i 1 e1
    :do (update-push-acontext ac :av "stage" "check 1"))
        (clear-update-eval-acontext ac :av "instance" e1))

(aspect "axsem" :type "if12else3" :stage "check 1" :value v1
    :acontext ac :agent ag :instance i :ap i 2 s2 :ap i 2 s3
    :match :p (mo "or1" (list (mo "1=2" (list v1 0))
                (mo "1=2" (list v1 0.0)) (mo "1=2" (list v1 "null")))
cond
        :p (clone-context ac) new-ac
        :ap new-ac "agent" new-ag
        :do (aset ag "precondition" (cons cond (aget ag
"precondition")))
            (aset new-ag "precondition"
                (cons (mo "not" (list cond)) (aget ag "precondition")))
            (clear-update-push-acontext ac :av "instance" s2)
            (clear-update-eval-acontext new-ac :av "instance" s3))
```

Семантика блока как пример шаблона проектирования "Итерация по стадии"

Lisp

```
(aspect "axsem" :type "{1}" :acontext ac :instance i :ap i 1 sts
    :match :av sts nil
    :exit (update-push-acontext ac :av "stage" "block iteration"
    :av "statements" sts :av "current" 0 :av "bound" (length sts)))

(aspect "axsem" :type "{1}" :stage "block iteration" :acontext ac
    :av "statements" sts :av "current" j :av "bound" n
```

```
    :match :v (< j n) T
    :exit (update-push-acontext ac :av "current" (+ j 1))
    (match :ap ac "stage" "block iteration" :ap ac "statements" sts
        (clear-update-eval-acontext ac :av "instance" (nth j sts))))
```

Текущее состояние проекта по созданию фрейворка для разработки формальных семантик и инструментов верификации на их основе

- Задействованные ЯП: C, C+ассемблерные вставки, C++, C#, Rust, Go, Python, Reflex, poST, Java, Javascript, Verilog
- Опубликованы первые статьи по проекту
- Реализуется интерпретатор языка